

```

1 begin
2     using Oscar: ZZ, QQ, factorial, binomial, combinations, permutations,
        power_series_ring, polynomial_ring, symmetric_group, coeff
3     using Latexify
4 end

```

WARNING: Imported binding Oscar.permutations was undeclared at import time during import to workspace#2.

(Univariate power series ring in x_ps with precision 100, x_ps + O(x_ps^101))
over rational field

```

1 begin
2     Z, (x, y, n, k, A, B, D) = ZZ[:x, :y, :n, :k, :A, :B, :D]
3     # Global Power Series Ring for generating functions (up to precision 100)
4     R_ps, x_ps = power_series_ring(QQ, 100, "x_ps")
5 end

```

LECTURE 6: The Principle of Inclusion-Exclusion

Scope: We know from the rule of sum that if A and B are sets with no elements in common, then $|A \cup B| = |A| + |B|$. But what if they have elements in common? Elements appearing in both sets are counted twice. To fix this, we subtract the overlap: $|A \cup B| = |A| + |B| - |A \cap B|$. This is the Principle of Inclusion-Exclusion (PIE).

I. Multiples and Sets

Consider a simple question: How many numbers from 1 to 10 are odd or a multiple of 3?

- Odd numbers (A): {1, 3, 5, 7, 9} (Size: 5)
- Multiples of 3 (B): {3, 6, 9} (Size: 3)
- Overlap ($A \cap B$): {3, 9} (Size: 2)

By PIE, the answer is $5 + 3 - 2 = 6$.

Let's compute how many numbers between 1 and 1000 are not divisible by 2, 3, or 5.

Divisibility Proof (Numbers 1 to 1000 not div by [2, 3, 5]):

- **Pure FP Filter:** 266
- **PIE Formula:** 266
- **Mathematical Proof Aggregation:** true

```
begin
  # CM for "Numbers between 1 and 1000 not divisible by 2, 3, or 5"

  # 1. Pure FP (Filter / Iteration)
  not_div_fp(n=1000, A=[2, 3, 5]) = length(filter(i -> all(d -> i % d != 0, A),
    1:n))

  # 2. PIE Combinatorial Formula (Stateless map/reduce over subsets)
  # n - (div by 1) + (div by 2) - (div by 3)...
  pie_not_div_by(n=1000, A=[2, 3, 5]) = n + sum(
    map(k -> (-1)^k * sum(map(combo -> n ÷ reduce(lcm, combo, init=1),
      collect(combinations(A, k)))), 1:length(A))
  )

  fp_ans_div = not_div_fp()
  pie_ans_div = pie_not_div_by()

  Markdown.parse("""
  ### Divisibility Proof (Numbers 1 to 1000 not div by [2, 3, 5]):
  * **Pure FP Filter**: $(latexify("$fp_ans_div", env=:inline))
  * **PIE Formula**: $(latexify("$pie_ans_div", env=:inline))

  * **Mathematical Proof Aggregation**: $(fp_ans_div == pie_ans_div)
  """)
end
```

```
(Set([3, 1, 5, 7, 9]), Set([3, 6, 9]), Set([3, 1, 5, 6, 7, 9]))
```

```
begin
  const set_ranges = 1:10
  const set_intersections = [3]

  A_set = Set(filter(n -> isodd(n), set_ranges))
  B_set = Set(filter(n -> all(d -> n % d == 0, set_intersections), set_ranges))

  A_set, B_set, union(A_set, B_set)
end
```

II. Poker Hands

How many 5-card poker hands have at least 1 card in each suit?

- Total hands: $\binom{52}{5}$
- Subtract hands missing 1 suit: $4 \times \binom{39}{5}$
- Add hands missing 2 suits: $\binom{4}{2} \times \binom{26}{5}$
- Subtract hands missing 3 suits: $\binom{4}{3} \times \binom{13}{5}$

Poker Hands Proof:

- **Pure FP (PIE Formula):** 685464
- **Abstract Algebra (Polynomial Coeff):** 685464
- **Group Theory (S_A Orbit):** 685464
- **Mathematical Proof Aggregation:** true

```
begin
# 1. Pure FP (Julia Base / PIE Formula)
poker_fp(n=52, A=4, k=5) = sum(
    map(i -> (-1)^i * Base.binomial(A, i) * Base.binomial(n - i * (n ÷ A), k), 0:
        (A-1))
)

# 2. Abstract Algebra (Polynomial Expansion in Oscar)
function poker_aa(n=52, A=4, k=5)
    cards_per_suit = n ÷ A
    # Polynomial for one suit: choose at least 1 card, up to 'k' cards
    P_suit = sum(map(i -> Base.binomial(cards_per_suit, i) * x^i, 1:k))
    return coeff(P_suit^A, x^k)
end

# 3. Group Theory (Symmetric Group Orbits for partitions)
function poker_group(n=52, A=4, k=5)
    cards_per_suit = n ÷ A
    # The only valid integer partition of k=5 into A=4 non-zero parts is [2, 1,
    1, 1]
    base_partition = [2, 1, 1, 1]

    # Generate the orbit of this partition under the symmetric group S_A
    G = symmetric_group(A)
    orbit = Set([ [base_partition[g(i)] for i in 1:A] for g in G ])

    # For each unique permutation in the orbit, calculate the combinations
    return ZZ(sum(map(part -> reduce(*, map(c -> Base.binomial(cards_per_suit,
        c), part)), collect(orbit))))
end

ans_poker_fp = poker_fp()
ans_poker_aa = poker_aa()
ans_poker_group = poker_group()

Markdown.parse("""
### Poker Hands Proof:
* **Pure FP (PIE Formula)**: $(latexify("$ (ans_poker_fp)", env=:inline))
* **Abstract Algebra (Polynomial Coeff)**: $(latexify("$ (ans_poker_aa)",
env=:inline))
* **Group Theory (S_A Orbit)**: $(latexify("$ (ans_poker_group)", env=:inline))

* **Mathematical Proof Aggregation**: $(ans_poker_fp == ans_poker_aa ==
ans_poker_group)
""")
end
```

III. Derangements

The mixed-up homework problem: How many ways can n assignments be returned to n students such that *no* student gets their own homework back? This is the derangement number D_n .

Formula derived via PIE: $D_n = n! \sum_{k=0}^n \frac{(-1)^k}{k!}$

Derangements Proof (5 items):

- **Pure FP (PIE Formula): 44**
- **Abstract Algebra (E.G.F.): 44**
- **Group Theory (Symmetric S_n Orbits): 44**
- **Mathematical Proof Aggregation: true**

```
begin
  # CM for "Derangements (The mixed-up homework problem)"

  # 1. Pure FP (Julia Base / PIE Formula)
  derangement_fp(n=5) = sum(map(k -> (-1)^k * (Base.factorial(n) ÷
    Base.factorial(k)), 0:n))

  # 2. Abstract Algebra (Exponential Generating Functions in Oscar)
  function derangement_aa(n=5)
    # E.G.F for Derangements: D(x) = exp(-x) / (1 - x)
    # Uses the globally defined power series ring x_ps
    D_x = exp(-x_ps) * inv(1 - x_ps)
    # Multiply coefficient by n! to get the combinatorial number
    return ZZ(coeff(D_x, n) * Base.factorial(n))
  end

  # 3. Group Theory (Symmetric Group Orbits with no fixed points)
  function derangement_group(n=5)
    G = symmetric_group(n)
    length(filter(g -> all(i -> g(i) != i, 1:n), collect(G)))
  end

  ans_der_fp = derangement_fp()
  ans_der_aa = derangement_aa()
  ans_der_grp = derangement_group()

  Markdown.parse("""
  ### Derangements Proof (5 items):
  * **Pure FP (PIE Formula)**: $(latexify("$$(ans_der_fp)", env=:inline))
  * **Abstract Algebra (E.G.F.)**: $(latexify("$$(ans_der_aa)", env=:inline))
  * **Group Theory (Symmetric  $S_n$  Orbits)**: $(latexify("$$(ans_der_grp)",
    env=:inline))

  * **Mathematical Proof Aggregation**: $(ans_der_fp == ans_der_aa == ans_der_grp)
  """)
end
```